Reading between the lines

An evidence-first candidate ranker for the Redrob Intelligent Candidate Discovery & Ranking Challenge

100,000 candidates · one Senior AI Engineer JD · 94 seconds · zero LLM calls at rank time

The pool is designed to fool keyword systems

What 30 minutes of data exploration told us before any code was written

68%

of the pool has non-engineering titles (HR, accounting, civil...) — bulk noise

~5,100

keyword stuffers: non-technical profiles listing RAG, Pinecone, FAISS as skills

~80

honeypots with subtly impossible profiles — >10% of them in a top-100 disqualifies

≤ 5 min

CPU-only, 16 GB, no network: no LLM-per-candidate architectures allowed

Plus: behavioral twins (identical on paper, different availability) and “plain-language Tier 5s” who never use a single buzzword.

Key insight: the traps live close to the JD in embedding space

The usual hybrid

Embed JD + profiles, rank by similarity, sprinkle rules on top.

A stuffer's skill list is engineered to sit next to the JD in vector space. Similarity-driven ranking maximizes exactly the failure mode this dataset punishes.

Our inversion

Explicit, JD-derived evidence reasoning drives the score.

Evidence is read from career-history prose — text someone had to write about real work — never from the self-reported skills list. Semantic similarity (TF-IDF) enters only as a 15% refinement on an already-vetted shortlist.

Architecture: a funnel that earns trust at every stage

100,000	raw pool	streamed JSONL, one pass
~32,000	title prescreen	the JD's own rule: non-engineering titles are out
~21,000	gates + evidence	honeypot gates · evidence from career prose · availability multiplier
1,500	shortlist refine	TF-IDF cosine vs JD, 15% blend — tie-breaker, never driver
100	ranked output	fact-grounded reasoning per candidate

Evidence comes from what they did, not what they claim

Nine concept lexicons applied to career-history descriptions and titles only

Retrieval (w 3.0)	embeddings, FAISS, vector DBs, semantic search, HNSW, dense retrieval
Ranking (w 3.0)	ranking models, recommenders, LTR, BM25, discovery feeds, search relevance
Evaluation (w 2.5)	NDCG, MRR, A/B tests, offline-online correlation, relevance judgments
Supporting (w 0.8–1.5)	LLM engineering · NLP/IR · production scale · XGBoost-LTR · open source · HR-tech

Context-weighted

- × recency decay on past roles
- × 0.55 if earned at a services firm
- × 1.25 conjunction bonus when retrieval AND ranking co-occur — the JD's ideal is the intersection, not the sum

Skills list used only for corroboration: a skill counts when the prose backs it.

How each trap dies

1 Keyword stuffers

≥5 AI skill claims with ≤1 corroborated in work history and no core evidence → ×0.10. Non-engineering titles never enter scoring at all (the JD's explicit rule).

2 Honeypots

Internal-consistency gates: job duration contradicting its own dates (>12 mo), claimed YoE exceeding the dated career span, “expert” skills with zero months of use. 65 profiles flagged; 0 honeypots in our top-100 even under tightened thresholds.

3 Behavioral twins

Availability is a multiplier on fit, not a feature: last-active decay, recruiter response rate, interview completion, notice period, location/visa. Identical resumes separate cleanly.

4 Plain-language Tier 5s

Lexicons include prose phrases (“built a recommendation system”, “discovery feed”, “collaborative filtering”) — candidates who never say RAG or Pinecone still surface.

Every weight traces to a JD sentence

docs/JD_MAPPING.md in the repo carries the full table — a sample:

"research-only careers — we will not move forward"	→	`research_only → ×0.15`
"under 12 months of LangChain calling OpenAI"	→	`shallow_llm → ×0.55`
"only worked at consulting firms in their entire career"	→	`services-only → ×0.30 (prior product experience passes, as the JD allows)`
"CV/speech/robotics without significant NLP/IR exposure"	→	`cv_primary → ×0.35`
"title-chasers switching every 1.5 years"	→	`hopper → ×0.75`
"hasn't written production code in 18 months"	→	`non-coding title → ×0.60`
"5–9 years… ideal 6–8"	→	`experience trapezoid, plateau 5–9, peak 6–8`

Results on the full 100K pool

94 s

full run, laptop CPU
(budget: 300 s)

~1.5 GB

peak RAM
(budget: 16 GB)

0

honeypot flags in top-100,
even with tightened checks

6.3 yrs

mean experience of top-100
(JD ideal: 6–8)

Sample reasoning (rank 42)

“Senior AI Engineer at Apple with 5.9 yrs; shipped ranking/recommendation systems (collaborative filtering, recommendation system) at Apple plus embeddings/vector-retrieval work, aligning well with the JD's production-retrieval focus. Strong availability signals: open to work, active this month, 80% recruiter response rate.”

92% of the top-100 are India-based · official validator: “Submission is valid.”

From hackathon to production

1. ## Learn the weights

The rubric is hand-tuned today; with recruiter-engagement labels it becomes a learning-to-rank model over the same interpretable features.

2. ## Precompute embeddings

Offline sentence-transformer index (allowed outside the 5-min window) upgrades the 15% lexical refinement to true semantic recall — gates still veto.

3. ## Close the loop

NDCG/MRR offline benchmarks + A/B hooks are the JD's weeks 9–12 mandate; the evidence features double as explanation strings recruiters can audit.

Repo: rank.py · ranker/ (5 modules) · tests · Streamlit sandbox · docs/JD_MAPPING.md